



Universidade Federal de Viçosa – Campus
UFV-Florestal
Ciência da Computação – Engenharia de Software II
Projeto Integrador 2025

Modelo de Classes - sprint 02

Designer de Sistemas

Equipe 03 - Jogo Desconecta

Ana Lúcia de Souza - 5084

Florestal

2025

Sumário

Back-end.....	3
Classes de Entidade (Entity):.....	3
Classes de Controle:.....	5
Classes de acesso ao Banco de Dados:.....	6
Relacionamento entre as classes:.....	7
Front-end.....	7

Back-end

A figura 1 mostra o Diagrama de Classes completo para os Casos de Uso da Sprint 02, isto é, **CSU01: JogarFase**. Observe que esse diagrama utiliza a categorização BCE (Boundary, Control, Entity), a qual implica que cada objeto é especialista em realizar um dos três tipos de tarefa, a saber: comunicar-se com atores (fronteira), manter as informações (entidade) ou coordenar a realização de um caso de uso (controle).

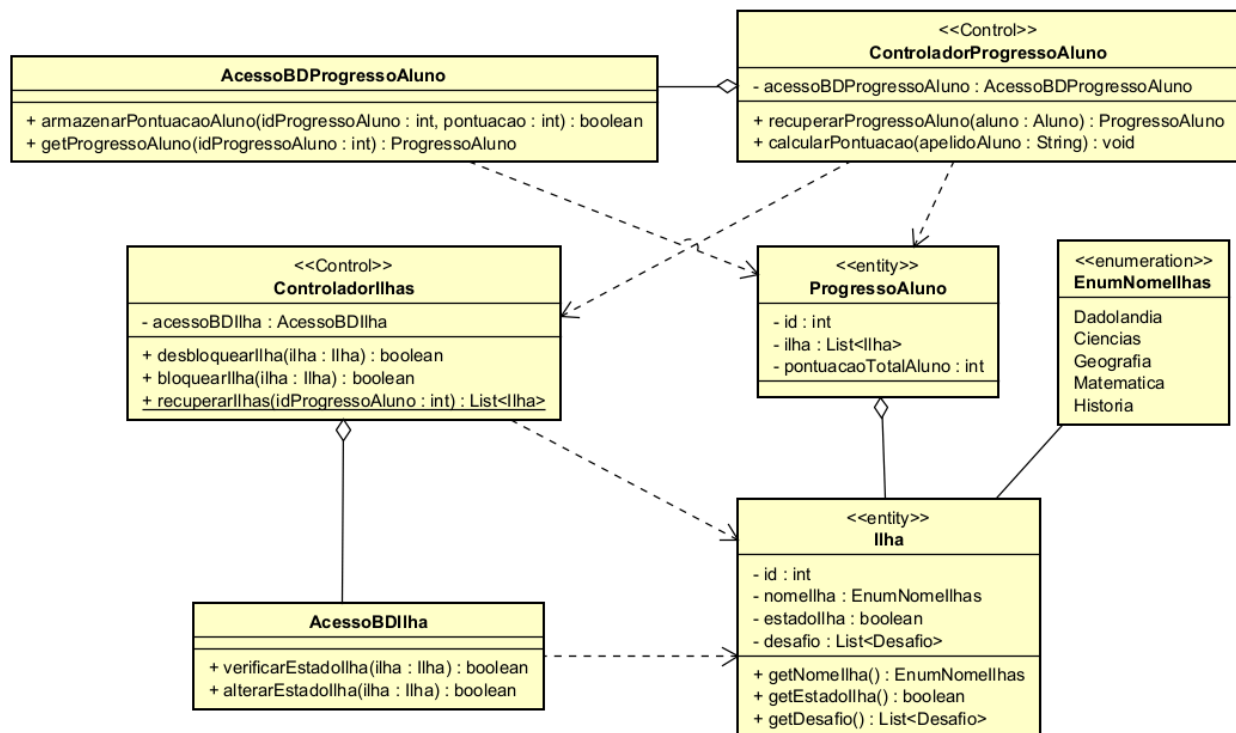


Figura 1. Diagrama de Classes para CSU01.

Classes de Entidade (Entity):

A seguir será explicada cada uma das classes de entidade:

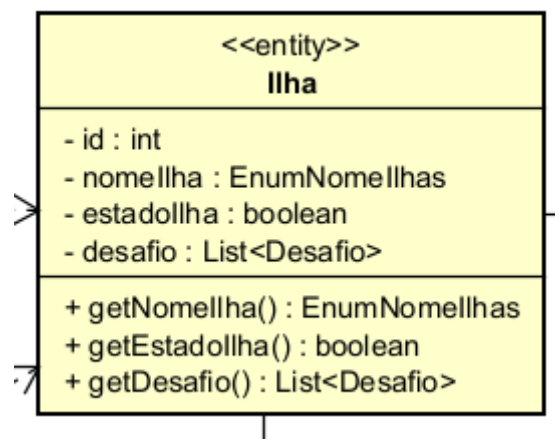


Figura 2. Classe Ilha.

- **Ilha:** Essa classe apresenta três atributos privados como mostrado na Figura 2. Observe que o atributo **desafio** é do tipo “**ListDesafio**” e como desafio não pertence a este caso de uso **não é necessário preocupar-se com ele neste momento**. Apenas declare a classe e a deixe vazia. Além disso, o atributo “**id**” foi colocado com o objetivo de facilitar as pesquisas por uma ilha jogada por um aluno específico. O “**nomellha**” representa o nome da ilha de conhecimento, ou seja, Dadolândia, Matemática, Ciências, etc, e deve ser um Enum, como pode ser visto na Figura 3. O atributo “**estadollha**” representa o “status” dessa ilha, ou seja, se essa ilha está desbloqueada (boolean: true) ou bloqueada (boolean: false). Observe que se a ilha estiver bloqueada ela não deve ser “clicável” no front-end, pois o Aluno não poderá ter acesso a ela no momento. Apenas os métodos getters foram adicionados, pois acredita-se que os setters seriam dispensáveis, mas eles podem ser implementados conforme a necessidade do programador.

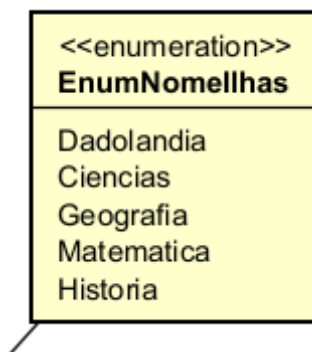


Figura 3. Enum para o nome das ilhas.

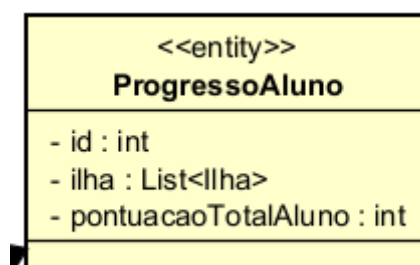


Figura 4. Classe ProgressoAluno.

- **ProgressoAluno:** Esta classe apresenta três atributos privados como mostrado na Figura 4. O atributo “**id**” foi colocado com o objetivo de facilitar a pesquisa pelo progresso de um aluno no banco de dados. O atributo “**ilha**” do tipo List<Ilha> representa todas as ilhas de conhecimento que o Aluno jogará, isso inclui a Dadolândia. O atributo “**pontuacaoTotalAluno**” representa a pontuação obtida pelo aluno em todos os desafios de todas as ilhas jogadas por ele até o momento.

Métodos getters e setters podem ser adicionados conforme a necessidade do programador .

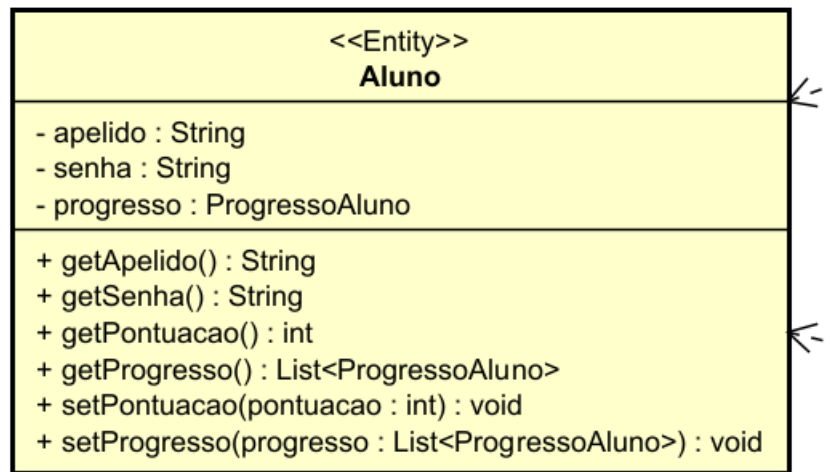


Figura 5 .Classe Aluno.

- Classe Aluno: Essa classe (Figura 5) **não pertence** a este caso de uso e **já foi implementada** nos casos de uso anteriores. Observe que houveram duas mudanças nesta classe: “progresso” agora é um atributo simples e não uma lista. Além disso, o atributo que representava a pontuação do aluno foi movido para a classe ProgressoAluno.

Classes de Controle:

A classe “**ControladorProgressoAluno**”, possui o atributo privado **acessoBDProgressoAluno**, que será criado pelo construtor dessa classe. Nesta classe também existem os seguintes métodos:

- **recuperarProgressoAluno(aluno : Aluno) : Progresso**: Esse método tem como objetivo retornar o progresso salvo do aluno enviado como parâmetro no banco de dados.
- **calcularPontuacao(apelidoAluno : String) : void**: Esse método calcula a pontuação atual do aluno e armazena no banco de dados usando o método **armazenarPontuacao** da classe **AcessoBDProgressoAluno**. Observe que o método “**calcularPontuacao()**” depende do caso de casos de uso ainda não implementados, sendo assim, não é necessário a sua implementação neste momento. Apenas deixe o corpo do método vazio.

A classe “**ControladorIlhas**”, possui o atributo privado **acessoBDIlha**, que será criado pelo construtor dessa classe. Nesta classe também existem os seguintes métodos:

- **desbloquearIlha(ilha : Ilha) : boolean**: Este método tem como objetivo alterar o estado da ilha (**estadollha**) para **true** no banco de dados, usando o método **alterarEstadollha** de **acessoBDIlha**.
- **bloquearIlha(ilha : Ilha) : boolean**: Este método tem como objetivo alterar o estado da ilha (**estadollha**) para **false** no banco de dados, usando o método **alterarEstadollha**.
- **recuperarIlhas(idProgressoAluno : int) : List<Ilha>**: Esse método retorna a lista de Ilhas de determinado ProgressoAluno, de acordo com seu “id”. Este método é estático, o que significa que não é necessário ter uma instância desta classe para fazer uma chamada do método.

Classes de acesso ao Banco de Dados:

A classe **AcessoBDProgressoAluno** será a responsável pelo acesso e manipulação do banco de dados referente à tabela de ProgressoAluno. Seus métodos serão descritos a seguir:

- **armazenarPontuacaoAluno(idProgressoAluno : int, pontuacao : int) : boolean**: Esse método recebe o id do ProgressoAluno e uma pontuacao, ambos inteiros, e salva o dado de pontuação na respectiva tabela.
- **getProgressAluno(idProgressoAluno : int) : boolean**: Esse método recebe o id do ProgressoAluno e retorna um objeto ProgressoAluno com todos os seus atributos preenchidos.

A classe **AcessoBDIlha** será a responsável pelo acesso e manipulação do banco de dados referente à tabela de Ilha. Seus métodos serão descritos a seguir:

- **verificarEstadollha(ilha : Ilha) : boolean**: Esse método recebe uma Ilha e realiza uma pesquisa na tabela, retornando o estado atualmente registrado na tabela.
- **alterarEstadollha(ilha : Ilha) : boolean**: Esse método recebe uma Ilha e altera o estado presente na tabela para o estado da ilha recebida como parâmetro.

Relacionamento entre as classes:

Observe que a classe `AcessoBDProgressoAluno` apresenta um relacionamento de agregação simples com `ControladorProgressoAluno`, sendo a primeira o objeto parte e a segunda o objeto todo. Em relação a `ControladorProgressoAluno`, há relações de dependência para com as classes `ControladorIlhas` e `ProgressoAluno`. Já `AcessoBDProgressoAluno` tem um relacionamento de dependência com `ProgressoAluno`.

Ademais, a classe `ControladorIlhas` possui uma relação de agregação simples com a `AcessoBDIlha`, sendo todo e parte, respectivamente. `ControladorIlhas` e `AcessoBDIlha` também têm uma relação de dependência com a classe `Ilha`. Por fim, a classe `Ilha` tem uma relação de agregação simples com `ProgressoAluno`, sendo a primeira o lado parte do relacionamento e a segunda, o todo.

Front-end

Para a modelagem do front-end, foi utilizado o diagrama de máquina de estados conforme apresentado na figura 6.

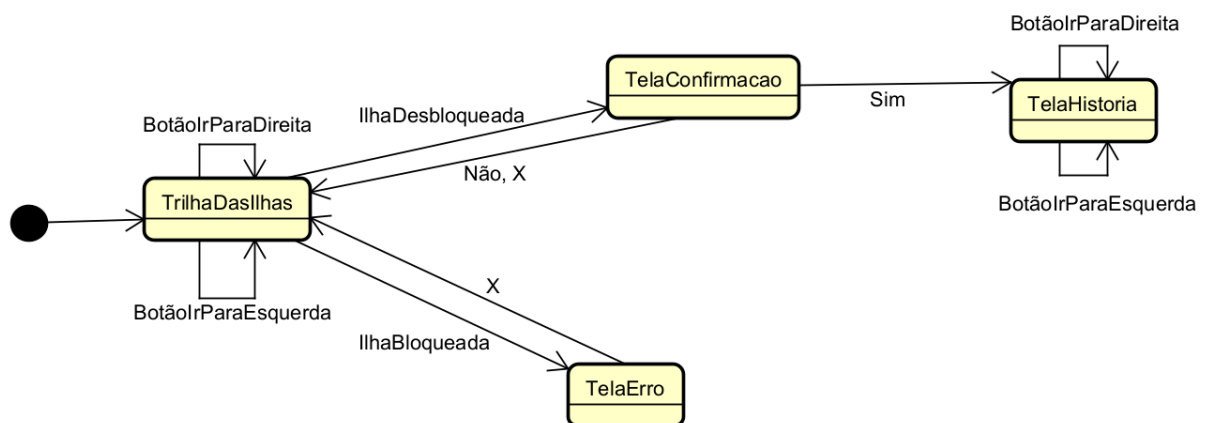


Figura 6. Modelagem front-end.

A figura 7 representa a “TrilhaDasIlhas” do programa. Ao clicar em “BotãoIrParaDireita”, o aluno é direcionado para a continuação desta tela e, ao clicar em “BotãoIrParaEsquerda”, a parte da tela que está à esquerda é mostrada”. Ao clicar em alguma ilha desbloqueada, a “TelaConfirmação” (Figura 8) é mostrada. Estando na “TelaConfirmação”, ao escolher “Não”, ou “X”, volta-se para “TrilhaDasIlhas”. Já se escolher a opção “Sim”, é mostrada a “TelaHistoria”(Figura 9) - observar no documento de protótipo que a história é composta por uma sequência de telas. Se em “TrilhaDasIlhas”

for escolhida alguma ilha bloqueada, a “TelaErro” (Figura 10) é mostrada. Se desta for escolhida a opção “X”, volta-se para a tela “TrilhaDasIlhas”.

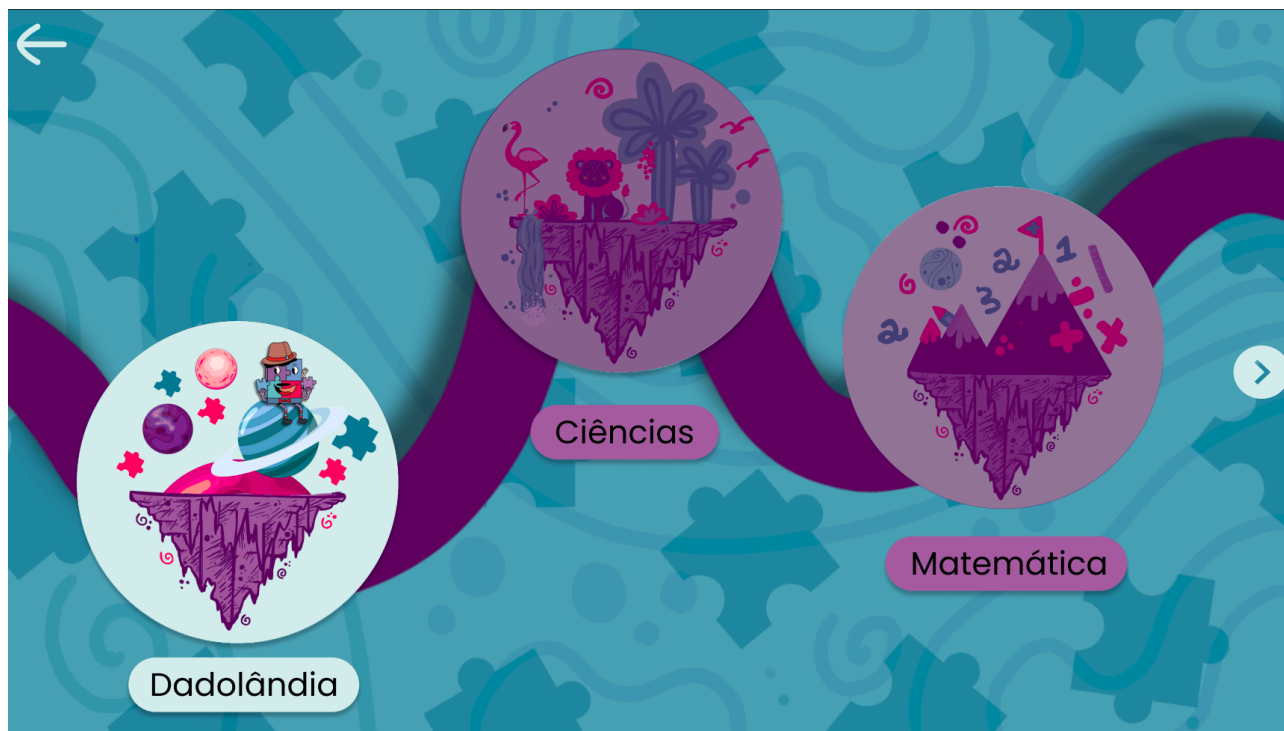


Figura 7. Prototipagem de TrilhaDasIlhas.

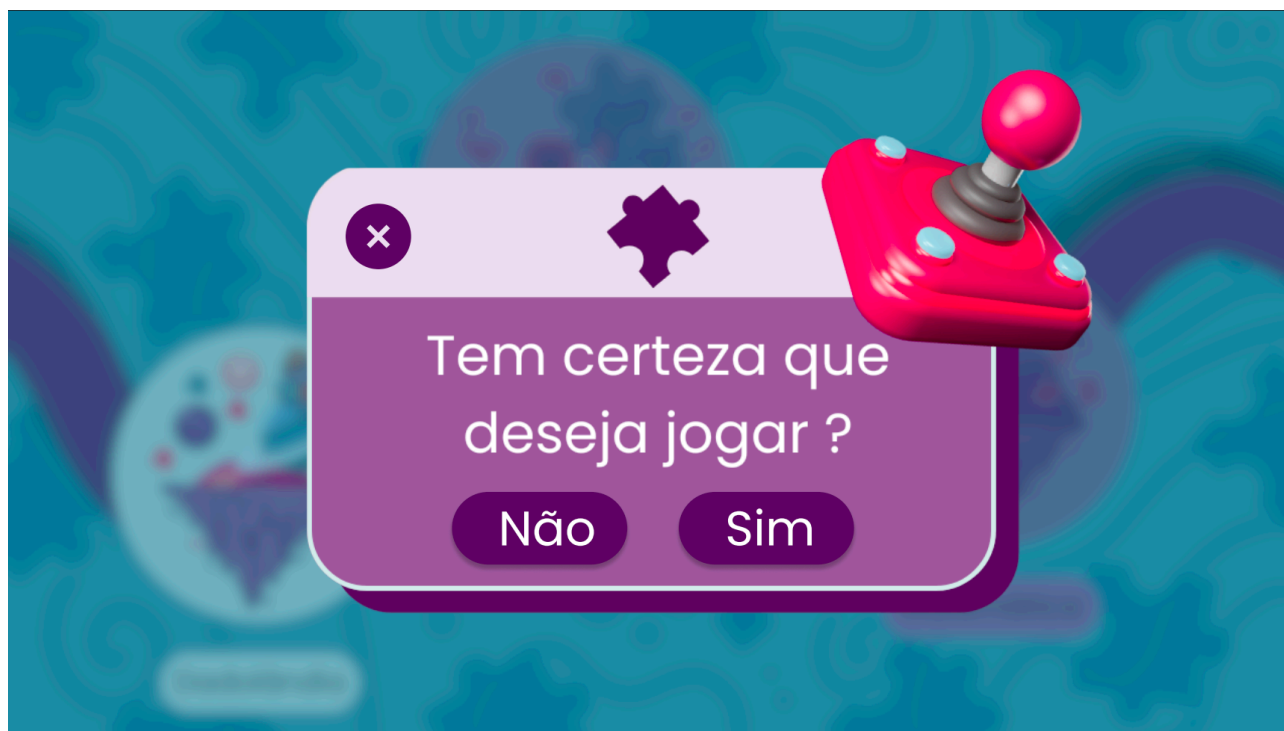


Figura 8. Prototipagem de TelaConfimacao.



Figura 10. Prototipagem de TelaHistoria.

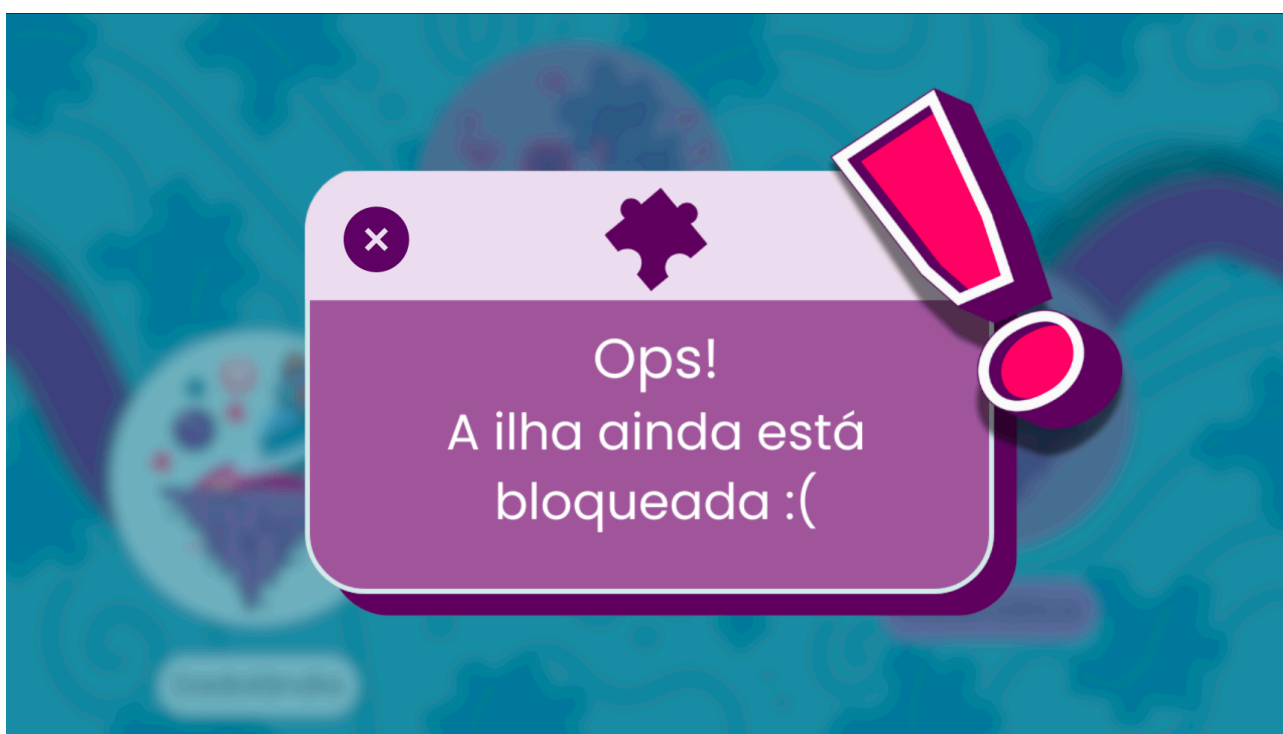


Figura 10. Prototipagem de TelaErro.